H

Hospital Management System

Project Transformation Report, Interview Guide & Learning Roadmap

Architecture | Security | Testing | CI/CD & Docker | UX & Design | Interview Prep

MERN stack: React 18 · Express · MongoDB

October 2026

Contents

0	Executive Summary	The one-page version
1	Project Overview	What it does, old vs new architecture
2	Complete Change Log	Every change, grouped by category
3	Before vs After	The most important code-level shifts
4	Interview Cheat Sheet	Concepts, answers, follow-ups
5	Learning Roadmap	What to study next, and why it's worth it
6	Recruiter View	How the project reads now
7	Portfolio Value	Resume, screenshots, demo script
8	Final Assessment	Scores and honest remaining gaps

> **How to use this document.** Read Section 0 and Section 3 first; they hold 80% of what you need for an interview. Use Section 4 the night before an interview. Use Section 5 to plan the next few weeks of study.

0 · Executive Summary

HMS started as a feature-rich but fragile MERN application: 15 modules and 7 roles, but no tests, no validation, a privilege-escalation bug at sign-up, and two competing visual styles. Three passes were made on it: **engineering hardening**, **UX and design**, and this **documentation** pass.

~4.0 → 6.3
Overall score (out of 10)

0 → 30
Automated tests (21 backend, 9 client)

247 → 0
Lint errors (18 warnings remain)

7
Critical security issues closed

1 pipeline
GitHub Actions CI: test, lint, build, Docker

Junior → SDE-1
Level the project now reads as

The five changes that matter most

Change	Why an interviewer cares
Role can no longer be chosen at sign-up	Before, anyone could register as `admin`. This is the classic "trust the client" mistake and the fastest way to fail a review.
Centralized error handling	Shows you understand Express middleware order, async errors, and not leaking internals.
Object-level authorization + field allowlists	Fixes IDOR and mass assignment, two OWASP Top 10 issues that most portfolio projects have.
Payment flow made tamper-proof and idempotent	Shows trust boundaries, constant-time comparison and idempotency, which are real fintech concerns.
Tests + CI + Docker	Turns "it works on my machine" into "every change is verified automatically".

> **Be honest in interviews.** Some work is partial: validation covers auth and payments only; the service layer has one service (`tokenService`); backend tests mock Mongoose (no real-DB tests); Docker images were written but not run; only the Patients list uses the new skeleton and empty states. Saying this unprompted reads as senior, not weak.

1 · Project Overview

What the application does

A role-based hospital platform. Seven roles (**admin, doctor, patient, receptionist, nurse, lab staff, pharmacist**) each see a tailored dashboard and navigation. Modules: appointments, patients, doctors, staff, departments, beds and admissions, electronic medical records (EMR), lab tests, pharmacy stock, billing with Razorpay online payment, reviews and notifications.

Technology stack

Layer	Technology	Added in this transformation
Frontend	React 18, Redux Toolkit, React Router 6, Axios, Vite	Axios interceptors, ErrorBoundary, design tokens, PageLayout, Toasts
Backend	Node.js, Express 4, Mongoose 9, JWT, bcryptjs	express-validator (now actually used), pino / pino-http
Security	helmet, express-rate-limit, express-mongo-sanitize, CORS	Algorithm pinning, timing-safe compare, regex escaping
Payments	Razorpay	Order-to-bill binding, idempotent verification
Testing	node:test (client)	Jest + Supertest (backend)
DevOps	Vercel (client only)	Dockerfiles, docker-compose, GitHub Actions

Original architecture

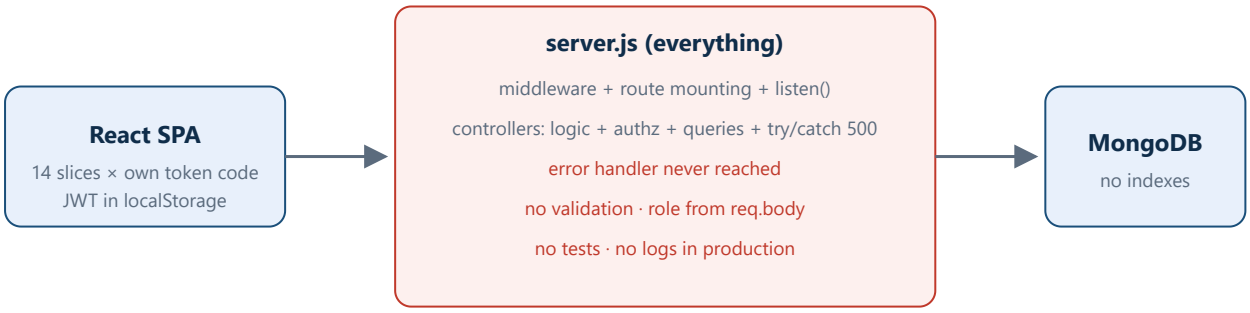


Figure 1. Before: a working app with no seams for testing, no input boundary and no observability.

New architecture

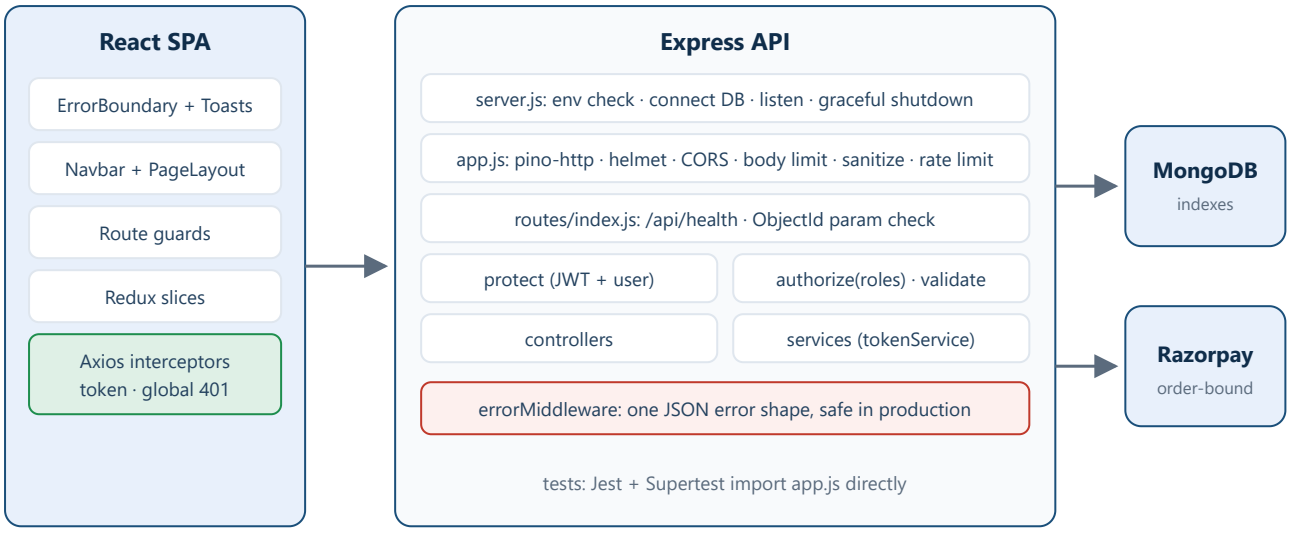


Figure 2. After: clear boundaries; each layer has one job and can be tested in isolation.

Request lifecycle (end to end)



A 401 anywhere comes back to the interceptor, which logs the user out once, globally.

Key engineering decisions

- **Split `app.js` from `server.js`.** Tests can import the app without opening a port or connecting to a database.
- **Validate at the edge, enforce in the database.** Request validators reject bad input early; schema rules and unique indexes are the last line of defence (for example, preventing double-booking).
- **Never trust client-supplied identity or money.** Role, doctor identity, bill totals and the bill being paid are all derived on the server.
- **Re-check the user on every request.** One extra indexed lookup makes deactivation and role changes take effect immediately.
- **Incremental UI system, not a rewrite.** Design tokens and shared layout first; existing screens keep working while they migrate.

2 · Complete Change Log

Each card follows the same structure: what existed, what exists now, why, and how to say it in an interview.

ARCHITECTURE

App / server split with a real bootstrap

`backend/app.js · backend/server.js · backend/config/env.js · backend/config/db.js`

Before	One `server.js` built the app, called `connectDB()` without awaiting it, and started listening immediately.
Now	`app.js` exports the Express app. `server.js` validates env, awaits the DB, listens, and shuts down gracefully on SIGTERM/SIGINT.
Why	The server could accept traffic before the DB was ready, and the app could not be tested without a live server.
Benefits	Testability, safe startup, clean container stops.
Recruiter impact	Shows you understand application lifecycle, not just routes.
Talking point	*"I separated composition from runtime so Supertest can import the app, and the process only accepts traffic once dependencies are healthy."*

Central route registry

`backend/routes/index.js`

Before	15 `require` and `app.use` pairs in `server.js`, inconsistently indented, including a commented-out duplicate.
Now	One map of path to router. ObjectId validation is attached to every router's `:id`-style params automatically.
Why	A cross-cutting rule (valid ids) should be applied once, not remembered per file.
Talking point	*"Cross-cutting concerns go in one place so new routes get them for free."*

Token service (first service-layer module)

`backend/services/tokenService.js · authController.js · authMiddleware.js`

Before	`jwt.sign` in the controller, `jwt.verify` in middleware, with no shared options.
Now	`signToken` / `verifyToken` with the algorithm pinned to HS256 and a configurable expiry.
Why	Security settings that live in two places drift apart.
Talking point	*"JWT policy lives in one module; changing expiry or algorithm is a one-line change."*

BACKEND

Centralized error handling and a standard error shape

`middlewares/errorMiddleware.js · utils/AppError.js · utils/asyncHandler.js · all 15 controllers`

Before	Every handler ended with `res.status(500).json({ message: error.message })`. The global handler existed but was unreachable. Bad ids returned 500.
Now	Controllers call `next(error)` or throw `AppError`. One handler maps CastError→400, ValidationError→400, duplicate key→409, JWT errors→401, and hides 500 messages in production.
Why	Leaking database messages helps attackers; wrong status codes break clients.
Benefits	Consistent `{ success:false, message, details? }`; less code per handler.
Recruiter impact	One of the first things reviewers check in an Express codebase.
Talking point	*"Controllers describe the happy path; one middleware decides status codes and what is safe to expose."*

Health endpoint

`routes/index.js · backend/Dockerfile (HEALTHCHECK)`

Before	`GET /` returned a static string, even when the database was down.
Now	`GET /api/health` returns uptime and DB state; 503 when Mongo is disconnected. It is excluded from rate limiting.
Talking point	*"Health reflects dependencies, so orchestrators restart or route away from a broken instance."*

Structured logging

`utils/logger.js · app.js · server.js`

Before	`console.log` gated on `NODE_ENV !== 'production'`, so production had no logs at all.
Now	pino JSON logs with a request id per request (honours `x-request-id`), and redaction of passwords and auth headers.
Talking point	*"Every log line has a request id, so I can follow one user's failing request across the system."*

Search that actually works

`patientController.js · appointmentController.js · billController.js`

Before	`$regex` on `'userId.name'`, a field that does not exist on the stored document, so it never matched. Appointments filtered in memory *after* pagination, so totals were wrong.
Now	Matching users are resolved first, then the query filters by their ids before skip/limit.
Talking point	*"MongoDB populate is a client-side join, so you cannot query a populated field. I resolve references first."*

DATABASE

Schema validation, timestamps and indexes

`models/User.js · Patient.js · Appointment.js · Bill.js`

Before	No indexes; no `timestamps`, yet code sorted by `createdAt`; mixed-case emails; password hash returned by default.
Now	Length, format and range rules; `lowercase` emails; `password select:false`; timestamps; indexes on common filters; unique `Patient.userId`.
Talking point	*"Validation at the API is for good error messages; validation in the schema is for data integrity."*

Double-booking prevention (race-free)

`models/Appointment.js`

Before	Two patients could book the same doctor at the same time.
Now	A partial unique index on `(doctorId, date, time)` where `status = 'Scheduled'`. Collisions return 409.
Why	A "check then insert" in code has a race window; the database constraint does not.
Talking point	*"I pushed the invariant into the database, so concurrent requests cannot violate it."*

SECURITY

Privilege escalation at registration

`authController.js · authRoutes.js · client AddStaffForm.jsx`

Before	`role: role || 'patient'`, so `{"role":"admin"}` created an admin.
Now	Public sign-up always creates a patient. Staff accounts use the admin-only `POST /api/auth/staff-accounts`.
Recruiter impact	Removes the single biggest red flag in the repository.
Talking point	*"Authorization data never comes from the client. Roles are assigned by a privileged path."*

Login and session hardening

`authController.js · authMiddleware.js · tokenService.js · models/User.js`

Before	404 "User not found" vs 400 "Invalid password" (user enumeration); `isActive` ignored; bcrypt cost 10; JWT role trusted for 24h.
Now	One generic 401; comparison against a dummy hash flattens timing; bcrypt 12; inactive users rejected; every request re-reads role and active state.
Talking point	*"Login doesn't reveal which emails exist, and deactivating a user takes effect on their next request."*

IDOR and mass assignment

`patientController.js · appointmentController.js · billController.js · labController.js · emrController.js · emrRoutes.js`

Before	Any patient could list all patients or read anyone's lab results. `findByIdAndUpdate(id, req.body)` let callers overwrite any field. A doctor could author an EMR as another doctor.
Now	Ownership checks for patient-role reads; explicit allowlists for updates; EMR doctor derived from the token; route roles aligned with controller rules.
Talking point	*"Role checks answer 'can you use this endpoint'; ownership checks answer 'can you touch this record'. You need both."*

Payment integrity

`paymentController.js · paymentRoutes.js · models/Bill.js`

Before	`billId` came from the client at verification, so one valid signature could mark a different bill paid. String `!==` comparison. Mode `'Online'` not in the schema enum.
Now	Order id stored on the bill at creation; verification looks the bill up by order id; `timingSafeEqual`; ownership check; idempotent; Razorpay public key returned to the client (fixes checkout not opening).
Talking point	*"The server is the source of truth for which bill and how much. The client only relays Razorpay's signed proof."*

Input validation and other hardening

`validators/authValidators.js · middlewares/validate.js · utils/escapeRegex.js · app.js`

Before	`express-validator` installed but unused; raw user input inside `$regex`; 10 MB body limit; plain-text rate-limit responses.
Now	Rules on auth and payment routes with field-level error details; ObjectId param checks; escaped and length-capped regex input; 100 KB limit; JSON 429 responses.
Talking point	*"Unescaped regex is a ReDoS and query-injection vector; I escape and cap it."*

FRONTEND

Global 401 handling and token attachment

`client/src/utils/setupAxios.js · main.jsx · authSlice.js`

Before	14 copies of `getToken()`; an expired token left the user on broken pages.
Now	One interceptor attaches the token and logs out on 401 (except on the login call itself). Validation details surface in auth errors.
Talking point	*"Auth policy is a cross-cutting client concern, so it lives in an interceptor, not in every API call."*

ErrorBoundary and real linting

`components/common/ErrorBoundary.jsx · .eslintrc.cjs · constants/theme.js`

Before	Any render error blanked the whole app; lint reported 247 errors, so nobody ran it.
Now	A fallback screen with reload; 0 lint errors (18 warnings), plus a real bug fixed (a duplicate object key).
Talking point	*"A linter that always fails is ignored. I made it pass, so CI can enforce it."*

UX & DESIGN

Design tokens and one visual language

`client/src/index.css · index.html · App.css (deleted) · 5 files with font @imports removed`

Before	Login and dashboards used DM Sans with one navy; other pages used Arial with another. Fonts imported separately per component.
Now	CSS variables for color, spacing, radius and shadow; shared `.btn`, `.surface`, `.page` classes; fonts loaded once with preconnect.
Talking point	*"Tokens first, components second: change the palette in one file and every screen follows."*

Navigation rewrite

`components/Navbar.jsx · Navbar.css`

Before	Showed `undefined - admin` after refresh; no active state; overflowed on mobile; several pages unreachable from the menu.
Now	Active-link indicator; hamburger menu under 1024px; avatar menu with profile and sign-out; complete per-role links; name loaded after refresh; Escape closes menus.
Talking point	*"Navigation is the most-used UI in the app, so I fixed discoverability and mobile first."*

Shared page layout, states and feedback

`PageLayout.jsx` · `TableSkeleton.jsx` · `EmptyState.jsx` · `Toast.jsx` · `StatusPage.jsx` · `NotFoundPage.jsx` · 7 pages · `PatientList.jsx` · `PaymentModal.jsx`

Before	Each page re-implemented its header inline; "⟳ Loading..." text; bare-text empty states; a plain `<h1>` 403 page; blank screen for unknown URLs; `alert()` for payment results.
Now	One layout with a `main` landmark; skeleton rows; empty states with a next action; branded 403/404 with a way back; non-blocking toasts.
Talking point	*"Every async screen has four states (loading, empty, error, success), and I designed all four."*

Accessibility pass

`index.css` · `Navbar.jsx` · `NotificationBell.jsx` · `LoginPage.jsx` · `PatientList.jsx`

Before	No skip link or focus ring; the notification bell was a clickable `div`; unlabeled inputs; tables without header scope.
Now	Skip link, `:focus-visible` ring, reduced-motion support, keyboard-operable bell and notifications, labelled inputs, error association, table captions and scopes, show/hide password.
Talking point	*"In healthcare, accessibility is often a legal requirement, so I treated keyboard and screen-reader use as features."*

TESTING

Backend unit and API tests

`backend/__tests__/api.test.js` · `unit.test.js` · `package.json`

Before	`"test": "echo Error: no test specified && exit 1"`.
Now	21 Jest + Supertest tests: health, 404, malformed JSON, validation, role escalation, duplicate→409, generic login error, missing/tampered token, deactivated user, RBAC, ObjectId check, forged signature, bill lookup by order id, error mapping, regex escaping, algorithm pinning.
Talking point	*"Each security fix has a regression test, so it can't silently come back."*

Client tests

`client/tests/setupAxios.test.mjs` · `client/package.json`

Now	Interceptor tests (token attached, logout on 401, no logout on failed login) alongside the existing auth-slice tests: 9 total.

CI/CD

GitHub Actions pipeline

`.github/workflows/ci.yml` · `root package.json`

Before	No CI. Root `lint` was `echo 'Linting...'`; obsolete conflicting cucumber packages.
Now	On push and PR: backend tests and audit; client lint, test and build; then Docker image builds. Root scripts delegate to real commands.
Talking point	*"Nothing merges unless tests, lint and build pass on a clean machine."*

DEVOPS

Docker and Compose

`backend/Dockerfile · client/Dockerfile · .dockerignore × 2 · docker-compose.yml`

Now	API: multi-stage, production dependencies only, non-root user, HEALTHCHECK. Web: Vite build served by nginx with SPA fallback. Compose: Mongo + API + web, API waits for a healthy Mongo, `JWT_SECRET` required.
Caveat	Written, not yet run end to end. Run `docker compose up --build` before claiming it in an interview.
Talking point	*"One command brings up the full stack with the same setup as CI."*

PRODUCTION READINESS

Operational safety and documentation

`server.js · config/env.js · README.md`

Now	Fail-fast env validation (32+ char `JWT_SECRET`); unhandled-rejection logging; shutdown timeout; no hard-coded production CORS fallback; README with architecture diagram, setup, design decisions, security notes and known gaps.
Talking point	*"A misconfigured deploy fails loudly at boot, not quietly at the first login."*

3 · Before vs After

1. Who decides the role?

BEFORE

```
const { role, ... } = req.body;
User.create({ ..., role: role || 'patient' });
```

AFTER

```
// public sign-up
createAccount(req.body, 'patient');
// admin-only route
router.post('/staff-accounts', protect,
  authorize('admin'), staffAccountRules,
validate, ...)
```

security · clear trust boundary

2. Error handling

BEFORE (×~70 HANDLERS)

```
} catch (error) {
  res.status(500).json({ message: error.message
});
}
```

AFTER

```
} catch (error) { next(error); }
// errorMiddleware.js
CastError → 400 · ValidationError → 400
11000 → 409 · JWT → 401 · 5xx hidden in prod
```

security · consistency · readability

3. Updating a record

BEFORE

```
Patient.findByIdAndUpdate(id, req.body, { new:
true })
```

AFTER

```
const updates = pick(req.body,
PATIENT_EDITABLE_FIELDS);
Patient.findByIdAndUpdate(id, { $set: updates },
  { new: true, runValidators: true })
```

security · data integrity

4. Verifying a payment

BEFORE

```
if (expected !== razorpay_signature) ...
Bill.findByIdAndUpdate(req.body.billId,
  { paymentStatus: 'Paid' })
```

AFTER

```
if (!timingSafeEqual(expected, sig)) throw 400
const bill = await Bill.findOne(
  { razorpayOrderId: razorpay_order_id });
assertCanPay(bill, req.user);
if (bill.paymentStatus !== 'Paid') { ...save }
```

security · idempotency · correctness

5. Authenticating a request

BEFORE

```
req.user = jwt.verify(token, SECRET);
// role trusted for 24h, any algorithm
```

AFTER

```
const decoded = verifyToken(token); // HS256 only
const user = await User.findById(decoded.id)
  .select('role isActive').lean();
if (!user?.isActive) throw new AppError(...,
401);
```

security revocation

6. Searching across a reference

BEFORE

```
{ 'userId.name': { $regex: search } } // never
matches
// appointments: filter after .limit()
```

AFTER

```
const users = await User.find({ name: rx });
filter.userId = { $in: users.map(u => u._id) };
// rx = escaped, capped input
```

correctness scalability security

7. Starting the server

BEFORE

```
connectDB();       // not awaited
app.listen(PORT);
```

AFTER

```
validateEnv();
await connectDB();
const server = app.listen(PORT);
on SIGTERM → server.close → mongoose.disconnect
```

reliability testability

8. UI shell and states

BEFORE

```
<Navbar />
<div style={{ padding: '30px' }}>
  <h2 style={{ color: '#2c3e50' }}>Patients</h2>
  {isLoading && '↻ Loading patients...'}
```

AFTER

```
<PageLayout title="Patients" subtitle="..."
  actions={<button className="btn btn-primary">}>
  {isLoading && <TableSkeleton columns={6} />}
  <EmptyState action={<Clear search />} />
```

maintainability consistency developer experience

4 · Interview Cheat Sheet

What is centralized error handling?

Short answer: One middleware turns every thrown or forwarded error into an HTTP response, so status codes and error shape are decided in one place.

How we used it: Controllers call `next(err)` or throw `AppError`; `errorMiddleware.js` maps Mongoose/JWT errors and hides 5xx details in production.

Why industry uses it: Consistent APIs, no leaked stack traces, a single place to log and alert.

Follow-ups: Why does Express need 4 arguments for error middleware? · How do async errors reach it in Express 4 vs 5? · Operational vs programmer errors?

Authentication vs authorization? What is IDOR?

Short answer: Authentication proves who you are; authorization decides what you may do. IDOR is when changing an id in the URL gives access to someone else's record.

How we used it: `protect` (authn) → `authorize(roles)` (route-level authz) → ownership checks in controllers (object-level authz).

Why industry uses it: Broken access control is #1 in the OWASP Top 10.

Follow-ups: RBAC vs ABAC? · Where would you centralize ownership policies? · How do you test an authz matrix?

How does JWT authentication work, and what are its weaknesses?

Short answer: The server signs a token containing claims; clients send it on each request; the server verifies the signature without a session store.

How we used it: HS256 pinned, 1-day expiry, and a per-request user lookup so revocation works.

Why industry uses it: Stateless scaling across services. Weaknesses: hard to revoke, and XSS can steal tokens from `localStorage`.

Follow-ups: Refresh tokens? · httpOnly cookies vs localStorage? · What is the `alg: none` attack?

Why validate input, and where?

Short answer: Never trust the client. Validate at the API edge for good errors, and in the schema/database for integrity.

How we used it: express-validator chains + a `validate` middleware; ObjectId param checks; Mongoose schema rules; field allowlists against mass assignment.

Follow-ups: express-validator vs Zod vs Joi? · Validation vs sanitization? · What is mass assignment?

What is rate limiting?

Short answer: Capping requests per client per time window to stop brute force and abuse.

How we used it: A global limiter, plus a strict one on login and register that skips successful requests; health checks excluded.

Follow-ups: Why does an in-memory limiter break with multiple instances? (Use Redis.) · Token bucket vs fixed window? · Per-IP vs per-account?

What makes logging production-grade?

Short answer: Structured (JSON), leveled, correlated by request id, and free of secrets.

How we used it: pino + pino-http, a request id per request, redaction of auth headers and passwords.

Follow-ups: Logs vs metrics vs traces? · What would you alert on? · How would you ship logs to ELK / Datadog?

Liveness vs readiness: what is a health check?

Short answer: An endpoint orchestrators poll to decide whether to restart an instance or send it traffic.

How we used it: `/api/health` returns 503 when Mongo is down; Docker HEALTHCHECK uses it.

Follow-ups: Should liveness depend on the DB? · How does Kubernetes use each probe?

Unit vs integration tests? What did you mock?

Short answer: Unit tests check one function in isolation; integration tests check pieces working together, here the full HTTP stack.

How we used it: Supertest drives real routes and middleware; Mongoose models are mocked with Jest. Next step: mongodb-memory-server for real queries.

Follow-ups: The test pyramid? · Risks of over-mocking? · How do you test race conditions?

What is CI/CD?

Short answer: CI verifies every change automatically; CD ships verified changes automatically.

How we used it: GitHub Actions runs backend tests, client lint/test/build, then Docker builds, on every push and PR.

Follow-ups: Caching dependencies? · Branch protection? · Where do secrets live? · Blue/green vs canary?

Why Docker? What is a multi-stage build?

Short answer: Containers package the app with its runtime so it behaves the same everywhere. Multi-stage builds keep build tools out of the final image.

How we used it: API image with production deps only, running as a non-root user; client built then served by nginx; Compose for the full stack.

Follow-ups: Image vs container? · Why non-root? · Layer caching order? · Compose vs Kubernetes?

Route guards and global 401 handling on the frontend?

Short answer: Guards hide screens the user can't use; interceptors handle cross-cutting HTTP concerns. Neither is security; the server is.

How we used it: `PrivateRoute` / `RoleRoute` plus an Axios interceptor that attaches the token and logs out on 401.

Follow-ups: Why are client guards not security? · How would refresh-token rotation fit into the interceptor?

Why separate app.js from server.js? What is a service layer?

Short answer: Separating composition from runtime enables testing. A service layer holds business logic so controllers only translate HTTP.

How we used it: Supertest imports `app.js`; `tokenService` is the first service. Most logic still sits in controllers, which is the next refactor.

Follow-ups: Controller vs service vs repository? · How does dependency injection help testing?

How do you prevent a race condition like double-booking?

Short answer: Enforce the invariant atomically in the database (unique constraint or transaction), not with a read-then-write in code.

How we used it: A partial unique index on doctor, date and time for scheduled appointments; duplicate key → 409.

Follow-ups: What is a partial index? · When do you need a MongoDB transaction? · Optimistic locking?

What is idempotency, and why does it matter for payments?

Short answer: Repeating an operation has the same effect as doing it once. Networks retry; payments must not double-apply.

How we used it: Verifying an already-paid bill returns success without changes; the bill is found by stored order id.

Follow-ups: Idempotency keys? · Why use webhooks as well as client callbacks? · What is a timing attack?

What is a design system, and how does accessibility fit?

Short answer: Shared tokens and components that make UI consistent by default. Accessibility is built into those components, not added per page.

How we used it: CSS variables, `.btn` / `.surface`, PageLayout, skeleton, empty state, toasts; skip link, focus ring, keyboard-operable controls.

Follow-ups: What is WCAG AA contrast? · How do you test with a screen reader? · Tokens vs component library?

5 · Learning Roadmap

Not lessons, just enough to make you curious. Suggested order: **Validation → Error middleware → JWT & refresh tokens → Testing → GitHub Actions → Docker → Service layer → React Query → Redis**.

> **About resources:** official docs are linked by name and URL. For YouTube and courses, the channel or author is named so you can search for the current version rather than follow a link that may have moved.

API Validation

Beginner

What: Declarative rules that reject malformed input before business logic runs.

Why it's cool: One schema can validate, type your code (Zod), and generate API docs.

Here: Stopped the admin-registration exploit and ObjectId 500s.

Industry: Every public API: Stripe, GitHub, Shopify.

Learn next: Zod schemas · schema-to-OpenAPI · validation vs sanitization

Docs: express-validator.github.io · zod.dev · OWASP Input Validation Cheat Sheet · Time: 1 week

Error Middleware

Beginner

What: A 4-argument Express middleware that turns errors into responses.

Why it's cool: Deletes hundreds of lines of try/catch boilerplate.

Here: Made the unreachable global handler the single error path.

Industry: Every Express, Nest or Fastify service.

Learn next: Express 5 async errors · RFC 9457 problem details · error monitoring (Sentry)

Docs: expressjs.com/en/guide/error-handling.html · Time: 2-3 days

JWT Authentication

Intermediate

What: Signed, self-contained tokens that prove identity without a session store.

Why it's cool: Paste a token into jwt.io and see your own claims, then learn why `alg: none` was a real exploit.

Here: Pinned algorithm, generic login errors, live user check.

Industry: Auth0, Firebase, AWS Cognito, most SPAs.

Learn next: claims (exp, iat, aud) · HS256 vs RS256 · JWKS · storage trade-offs

Docs: jwt.io/introduction · OWASP JWT Cheat Sheet · Time: 1 week

Refresh Tokens

Intermediate

What: Short-lived access tokens plus a long-lived refresh token in an httpOnly cookie.

Why it's cool: Rotation detects stolen tokens; it's how Google keeps you signed in safely.

Here: The next fix: move the token out of `localStorage` and plug refresh into the interceptor.

Industry: OAuth 2.0 everywhere.

Learn next: rotation and reuse detection · SameSite cookies · CSRF

Docs: auth0.com/docs (refresh token rotation) · OAuth 2.0 RFC 6749 · Time: 1 week

Unit Testing

Beginner

What: Fast tests of one function or module in isolation.

Why it's cool: Refactor fearlessly; a red test tells you exactly what broke.

Here: Error mapping, regex escaping, token algorithm pinning.

Industry: A merge requirement at most product companies.

Learn next: mocks vs stubs vs spies · coverage · TDD

Docs: jestjs.io · Blog: Martin Fowler, "The Practical Test Pyramid" · Course: Kent C. Dodds, Testing JavaScript · Time: 1-2 weeks

Integration Testing

Intermediate

What: Tests that exercise real HTTP, middleware and (ideally) a real database.

Why it's cool: Catches the bugs unit tests can't, like the search query that never matched.

Here: 15+ Supertest tests through the full middleware stack.

Industry: API contract tests in every backend team.

Learn next: mongodb-memory-server · Testcontainers · fixtures and factories

Docs: github.com/ladjs/supertest · Time: 1 week

GitHub Actions / CI/CD

Beginner

What: YAML workflows that run on every push or PR.

Why it's cool: Free automation: tests, deploys, even dependency updates.

Here: Test → lint → build → Docker on every change.

Industry: GitHub Actions, GitLab CI, Jenkins, CircleCI.

Learn next: matrix builds · caching · environments and secrets · deploy jobs · branch protection

Docs: docs.github.com/actions · Blog: Martin Fowler, "Continuous Integration" · Time: 3-5 days

Docker

Intermediate

What: Package an app and its runtime into an image that runs identically anywhere.

Why it's cool: `docker compose up` and a new teammate is running in minutes.

Here: Multi-stage images, non-root user, health checks, Compose stack.

Industry: The unit of deployment for Kubernetes, ECS and Cloud Run.

Learn next: layer caching · image size · volumes and networks · Kubernetes basics

Docs: docs.docker.com · YouTube: TechWorld with Nana (Docker tutorial) · Time: 1-2 weeks

Service Layer Pattern

Intermediate

What: Business logic in plain modules; controllers only translate HTTP.

Why it's cool: The same logic can serve REST, a queue worker or a CLI.

Here: `tokenService` is step one; appointments and billing are next.

Industry: NestJS, Spring and .NET all build on this.

Learn next: thin controllers · domain errors · transactions in services

Blog: martinfowler.com/eaaCatalog/serviceLayer.html · Time: 1 week

Repository Pattern

Intermediate

What: An interface over data access, so services don't call Mongoose directly.

Why it's cool: Swap Mongo for an in-memory fake in tests, with zero mocking libraries.

Here: Would replace the `jest.mock('../models/...')` approach.

Industry: Domain-driven design and clean architecture codebases.

Learn next: clean / hexagonal architecture · unit of work

Blog: martinfowler.com/eaaCatalog/repository.html · Time: 3-5 days

Dependency Injection

Advanced

What: Pass dependencies in, rather than `require`-ing them inside.

Why it's cool: The secret behind easily testable frameworks like NestJS and Angular.

Here: Would let tests inject a fake Razorpay client instead of patching modules.

Industry: Spring, .NET, NestJS, Angular.

Learn next: factory functions · IoC containers · composition root

Blog: martinfowler.com/articles/injection.html · Docs: docs.nestjs.com/providers · Time: 1 week

Rate Limiting

Intermediate

What: Bounding how often a client can call you.

Why it's cool: The algorithms (token bucket, sliding window) are classic system-design interview material.

Here: Global and auth limiters with JSON 429s.

Industry: Every public API; Cloudflare and API gateways.

Learn next: algorithms · distributed limits with Redis · per-user quotas

Docs: express-rate-limit (npm) · Blog: Cloudflare and Stripe engineering blogs on rate limiting · Time: 3-5 days

Redis & Caching

Intermediate

What: An in-memory data store for caches, sessions, rate limits and queues.

Why it's cool: Sub-millisecond reads; one tool unlocks five features.

Here: Needed for a shared rate limiter across instances, and to cache the per-request user lookup.

Industry: Twitter/X timelines, GitHub, almost every high-traffic backend.

Learn next: cache-aside · TTL and invalidation · pub/sub · BullMQ queues

Docs: redis.io/docs · Free course: Redis University · Time: 1-2 weeks

Logging & Monitoring

Intermediate

What: Logs, metrics and traces: the three pillars of observability.

Why it's cool: Find a production bug from a dashboard instead of guessing.

Here: pino structured logs with request ids and redaction.

Industry: Datadog, Grafana, New Relic, OpenTelemetry.

Learn next: OpenTelemetry · Prometheus metrics · SLOs and alerts · Sentry

Docs: getpino.io · opentelemetry.io/docs · Book: Google SRE Book (sre.google, free online) · Time: 2 weeks

React Query (TanStack Query)

Intermediate

What: A library for server state: fetching, caching, retries, invalidation.

Why it's cool: It could delete most of the 14 Redux slices and the `?all=true` workaround.

Here: The natural next frontend refactor.

Industry: The default for server state in modern React apps.

Learn next: query keys · mutations and invalidation · optimistic updates · RTK Query comparison

Docs: tanstack.com/query · Blog: TkDodo's blog (React Query maintainer) · Time: 1 week

Design Systems

Intermediate

What: Tokens + components + guidelines that make UI consistent by default.

Why it's cool: Lets small teams ship polished UI fast, the way Stripe and Linear do.

Here: CSS tokens, shared buttons and surfaces, PageLayout, state components.

Industry: Material, Atlassian Design System, Primer (GitHub), Polaris (Shopify).

Learn next: component APIs · theming and dark mode · Storybook · headless UI (Radix)

Docs: atlassian.design · primer.style · storybook.js.org · Time: 2 weeks

Accessibility (a11y)

Beginner

What: Making UI usable by keyboard, screen readers and people with low vision.

Why it's cool: Try using your app with only Tab and Enter; it changes how you build.

Here: Skip link, focus ring, keyboard-operable bell, labelled forms, table semantics.

Industry: Legally required in healthcare and government; part of FAANG front-end interviews.

Learn next: WCAG 2.2 AA · ARIA patterns · focus management in modals · axe and Lighthouse

Docs: web.dev/learn/accessibility · w3.org/WAI/ARIA/apg · Time: 1-2 weeks

Database Indexing & Constraints

Intermediate

What: Data structures that make queries fast and rules the DB enforces atomically.

Why it's cool: One index can turn a 2-second query into 2 ms, and one constraint can remove a race condition.

Here: Compound indexes; partial unique index for booking.

Industry: Daily work for every backend engineer.

Learn next: `explain()` · ESR rule · transactions · text and Atlas Search

Docs: mongodb.com/docs/manual/indexes · Free course: MongoDB University · Time: 1-2 weeks

6 · Recruiter View

Before: concerns

- Anyone could self-register as admin
- Patients could read other patients' data
- Payment could be replayed onto another bill
- No tests; `npm test` exited with an error
- Fake lint script; no CI; no Docker
- No validation despite the library being installed
- No production logging
- Search features that could never work
- Inconsistent UI; dead-end error screens
- ~20 auto-generated "COMPLETE" reports overstating quality

After: resolved

- Role assigned server-side; admin-only staff provisioning
- Ownership checks and field allowlists
- Order-bound, idempotent, timing-safe payments
- 30 automated tests, including security regressions
- Real lint, CI pipeline, Dockerfiles and Compose
- Validation on auth/payment plus schema rules
- Structured logs with request ids
- Working search before pagination
- Shared design system, accessible navigation, 403/404
- *Still open:* the report files in the repo root

What level does the project read as now?

Level	Fit	Reason
Intern	Exceeds	Breadth alone clears this bar.
Junior Developer	Exceeds	Security, tests and CI are beyond typical junior portfolios.
SDE-1	**Solid fit**	Shows the fundamentals (trust boundaries, error design, testing, CI, Docker) and can explain trade-offs. This is the target level.
SDE-2	Partial	Missing: real-DB integration tests, a service/repository layer throughout, refresh tokens, webhooks, observability beyond logs, measured performance.
Senior	Not yet	Would need system-level design: scaling, multi-tenancy, audit trails for medical data, SLOs, incident readiness.

The fastest route to SDE-2 perception: add mongodb-memory-server tests, move appointments and billing into services, implement refresh tokens and a Razorpay webhook, and remove the generated report files from the repo root.

7 · Portfolio Value

Resume bullets (pick 3-4)

- Built a role-based hospital management platform (React, Redux Toolkit, Express, MongoDB) serving 7 roles across 15 modules, including Razorpay payments.
- Closed critical access-control flaws (privilege escalation, IDOR, mass assignment) and added regression tests for each fix.
- Hardened payment verification with server-side order binding, constant-time signature checks and idempotent processing.
- Introduced centralized error handling, request validation, structured logging and health checks; added Jest/Supertest suite and GitHub Actions CI.
- Containerized the stack with multi-stage Docker builds and Compose; built a token-based UI system with accessible navigation and loading/empty/error states.

Only claim what you can demo and explain. Run the Docker stack before listing it.

Screenshots to include

Screen	What it proves
Login page	First impression and form polish
Admin dashboard	Data-heavy UI and information hierarchy
Patients list: data, skeleton, empty state	You design all states, not just the happy path
Mobile navigation open	Responsive design
Payment flow (Razorpay test mode)	Third-party integration
Green GitHub Actions run	Engineering discipline
Terminal: `npm test` passing	Real tests exist

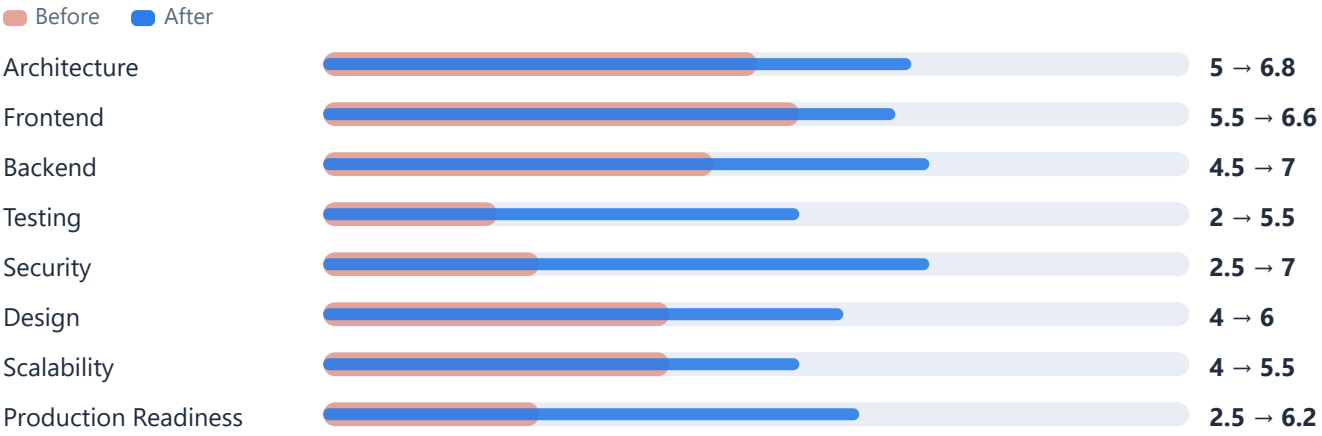
Live demo script (5 minutes)

1. Register as a patient → book an appointment → try booking the same slot again (409 message).
2. Log in as admin → create a nurse via staff provisioning → show role-specific navigation.
3. Pay a bill in Razorpay test mode → toast confirmation → bill shows Paid.
4. Show `curl /api/health`, then a request with `"role":"admin"` at sign-up still creating a patient.
5. Open the CI run and one security test, and explain what it protects.

Architecture decisions worth discussing

- Why the database enforces double-booking rather than application code.
- Why the server re-reads the user on every request, and what you would cache with Redis to make it cheaper.
- Why client route guards are UX, not security.
- The trade-off between Redux slices and React Query for server state.
- What you would change for real medical data: audit logs, encryption, soft deletes, retention.

8 · Final Project Assessment



~4.0	6.3 / 10	SDE-1
Overall before	Overall after	Level the project now reads as

Honest remaining gaps

Gap	Impact	Next step
Validation only on auth and payment routes	Other endpoints rely on schema rules	Add rule sets per resource
Tests mock Mongoose	Query bugs not caught	mongodb-memory-server integration tests
Logic still in controllers	Harder to reuse and test	Services for appointments and billing
JWT in localStorage, no refresh	XSS token theft risk	httpOnly cookie + rotation
No Razorpay webhook	Payment lost if the tab closes	Signed webhook endpoint
Some searches still query populated paths	EMR, admission, review and doctor name search ineffective	Apply the resolve-ids pattern
In-memory rate limiter	Breaks with multiple instances	Redis store
Most lists, forms and dashboards still inline-styled; 14 `window.confirm` dialogs	Visual inconsistency; no dark mode	Migrate to tokens; shared table and confirm modal
Docker stack not yet run	Unverified claim	`docker compose up --build`
Existing mixed-case emails	Those users can't log in after lowercasing	One-off migration script
~20 generated report files in repo root	Recruiter red flag	Remove or move to a docs archive

> **Bottom line.** The project moved from "impressive feature list, unsafe to run" to "credible SDE-1 portfolio piece with clear engineering judgement". The remaining gaps are exactly the right talking points for "what would you do next?", one of the most common closing interview questions.